Software Requirements Specification

for

BrowseBlock

AI browser extension for real-time browser based attacks detection and prevention.

Team 16

Prepared by,

Deon George(SJC23CS086)

Amal Kurian Royce (SJC23CS037)

Maria Royce (SJC23CS159)

Melbin K Thomas (SJC23CS164)

St. Joseph's College of Engineering and Technology, Palai

Date: 11/10/2026

Revision History

Name	Date	Reason for Changes	Version
BrowseBlock Project Team	11/10/2026	Initial SRS draft	1.0

Table Of Contents

Revision History i

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 Document Conventions . . . 1
- 1.3 Intended Audience and Reading Suggestions . . . 1
- 1.4 Product Scope . . . 1

2 Overall Description 3
- 2.1 Product Perspective . . . 3
- 2.2 Product Functions . . . 3
- 2.3 User Classes and Characteristics . . . 3
- 2.4 Operating Environment . . . 4
- 2.5 Design and Implementation Constraints . . . 5
- 2.6 User Documentation . . . 5
- 2.7 Assumptions and Dependencies . . . 5

3 External Interface Requirements 6
- 3.1 User Interfaces . . . 6
- 3.2 Hardware Interfaces . . . 6
- 3.3 Software Interfaces . . . 6
- 3.4 Communications Interfaces . . . 6

4 System Features 7
- 4.1 URL and Fake QR Phishing . . . 7
- 4.2 Cross-Site Scripting (XSS) . . . 7
- 4.3 Credential Harvesting . . . 9
- 4.4 HTML/JavaScript Redirects . . . 10
- 4.5 Clickjacking . . . 11
- 4.6 Cryptojacking . . . 11
- 4.7 Malicious File Download Detection . . . 12
- 4.8 Email Phishing Detection . . . 14
- 4.9 Browser Notification Scams . . . 16
- 4.10 Browser Extension Abuse . . . 16

5 Other Nonfunctional Requirements 18
- 5.1 Performance Requirements . . . 18
- 5.2 Safety Requirements . . . 18
- 5.3 Security Requirements . . . 18
- 5.4 Software Quality Attributes . . . 18

6 Other Requirements 19

A Glossary **20**

B Analysis Models **21**

C To Be Determined List **22**

Chapter 1

Introduction

1.1 Purpose

BrowseBlock is an AI/ML-powered browser security platform consisting of a browser extension and web dashboard. It detects, analyzes, explains, and helps prevent browser-based cyber threats in real time. This Software Requirements Specification defines the system's objectives, scope, interfaces, functional requirements, constraints, and quality requirements for development, testing, and evaluation.

1.2 Document Conventions

This document follows a structured Software Requirements Specification format. Mandatory requirements use the phrase "The system shall". Desirable capabilities use "should". Requirements are identified with unique BrowseBlock requirement identifiers, and detection results use the risk levels Safe, Suspicious, and High-risk.

1.3 Intended Audience and Reading Suggestions

This document is intended for project guides, review committee members, developers, testers, security analysts, and other stakeholders involved in BrowseBlock. Readers should begin with the Introduction and Overall Description, followed by External Interface Requirements and System Features. The remaining chapters describe non-functional requirements, other requirements, and supporting research material.

1.4 Product Scope

The browser extension acts as the primary protection layer by monitoring supported browsing activities, evaluating URLs, inspecting webpage content and behaviour, identifying suspicious redirects, and monitoring download events. It provides protection against ten threat categories: URL and fake QR phishing, Cross-Site Scripting (XSS), credential harvesting, HTML/JavaScript redirects, clickjacking, cryptojacking, malicious file downloads, email phishing, browser notification scams, and browser extension abuse.

The URL phishing detection module uses XGBoost to classify URLs based on extracted URL and domain features. For QR-based phishing detection, the system decodes supported QR-code inputs and evaluates the embedded URLs using the URL-analysis module.

The email phishing detection module uses Random Forest-based screening and available email-authentication signals within the browser extension for initial threat assessment. For detailed analysis through the web dashboard, a Retrieval-Augmented Generation (RAG) pipeline analyzes the email content, extracts URLs, and identifies suspicious patterns. The extracted URLs are further evaluated using BrowseBlock's URL phishing detection module. This two-stage approach enables lightweight, real-time screening in the browser extension while providing deeper semantic analysis and comprehensive risk assessment through the web dashboard when required.

Other protection modules analyze webpage scripts and behaviour, suspicious form submissions, redirect chains, framing patterns, notification-related scam indicators, and observable browser extension activity. Potentially malicious downloads in supported formats, including .zip, .pdf, .exe, .doc, and .docx, are assessed using file-type-specific static analysis, machine-learning models, and security rules. Untrusted files are not executed simply for analysis.

The browser extension remains lightweight and event-driven, while computationally intensive analysis is handled by the FastAPI backend where appropriate. The extension provides warnings or blocks risky activities when supported by browser APIs, permissions, and available evidence. These mechanisms provide an additional security layer but cannot guarantee prevention of every browser-based attack.

The React web dashboard enables users to manually scan URLs, QR codes, emails, and supported files; review detection history; examine threat reports and analytics; and understand the reasons, indicators, and evidence behind security decisions. Scan results, detection history, threat reports, and applicable system records are stored in MongoDB so that authorized users can retrieve relevant information beyond the current session. Access to stored information shall be restricted according to the authenticated user's identity and permissions.

The primary objective of BrowseBlock is to provide a unified, explainable, and user-friendly browser security solution for individuals and organizations. It aims to reduce exposure to phishing attacks and malicious web activities without replacing existing antivirus software, browser security features, or enterprise security solutions.

Chapter 2

Overall Description

2.1 Product Perspective

BrowseBlock combines a browser extension, a FastAPI backend, a React web dashboard, Firebase Authentication, MongoDB, machine-learning models, and AI-assisted analysis. The extension provides lightweight, event-driven protection during browsing, while the backend performs centralized processing and the dashboard supports manual scanning, reporting, and review.

2.2 Product Functions

The main functions include real-time URL and QR phishing detection, webpage and script inspection, credential-harvesting detection, redirect-chain analysis, clickjacking detection, cryptojacking detection, malicious-download analysis, email phishing analysis, browser-notification scam detection, browser-extension abuse detection, risk-based warnings and blocking, explainable results, history, and authorized security reporting.

2.3 User Classes and Characteristics

Primary Users: General internet users, students, employees, and individuals who browse websites, access web-based email services, encounter QR codes, and download files. These users may have limited cybersecurity knowledge and require clear warnings, understandable explanations, and practical security recommendations.

Organizational Users: Small businesses, educational institutions, and other organizations seeking an additional layer of protection against phishing emails, fraudulent websites, malicious QR codes, suspicious webpage behaviour, and unsafe downloads.

Security Analysts: Authorized users who review suspicious activities, examine detection evidence, interpret risk assessments, review threat reports, and investigate security incidents through the web dashboard.

System Administrators: Personnel responsible for managing system configurations, maintaining backend services, monitoring system performance, managing authentication-related configuration, and ensuring secure, authorized access to stored security records.

The system is intended to provide an accessible interface for non-technical users while offering detailed threat information and detection evidence for users who require more comprehensive security analysis.

2.4 Operating Environment

Browser Environment: Google Chrome and compatible Chromium-based browsers that support the required browser-extension APIs. Manifest V3 is the intended extension architecture, subject to the availability and limitations of the required APIs.

Web Dashboard: A responsive React-based web application accessible through modern desktop web browsers.

Frontend Technologies: React and Tailwind CSS for the dashboard interface.

Backend Environment: Python with FastAPI for REST API development, request validation, scan-request processing, coordination of detection modules, and communication with machine-learning and AI services. FastAPI endpoints expose the required backend functionality to the browser extension and web dashboard. The backend shall verify Firebase Authentication ID tokens for protected requests, enforce authorization, and interact with MongoDB for authorized data operations. The Firebase Admin SDK may be used for server-side token verification. Asynchronous request handling may be used where appropriate for I/O-bound operations.

Machine-Learning Environment: Python-based machine-learning components, including XGBoost for URL phishing detection and Random Forest for initial email phishing screening. Type-specific static analysis, machine-learning models, and deterministic security rules support malicious file assessment. SHAP is used where appropriate to explain relevant tree-model predictions.

AI-Based Email Analysis: RAG and LLM techniques support deeper semantic analysis of suspicious emails, including social engineering, impersonation, urgency, credential harvesting, and phishing intent. Available email-authentication signals are incorporated into the overall risk assessment where applicable.

Database Environment: MongoDB is used as the application's database for storing relevant user profile metadata, scan history, detection results, threat reports, and system records. Firebase Authentication manages user authentication and credentials; passwords and authentication secrets shall not be stored in MongoDB. Application data shall be associated with the appropriate authenticated user or authorized role, and database access shall be restricted accordingly.

Network Requirements: Network connectivity is required for communication between the browser extension, the FastAPI backend, the web dashboard, Firebase Authentication, MongoDB, and any enabled external threat-intelligence or AI services. Access to external services depends on their configuration and availability. MongoDB access shall be restricted to authorized backend services rather than exposed directly to untrusted clients.

Security Requirements: The system should use HTTPS for network communication, Firebase Authentication for user sign-in and identity verification, server-side verification of Firebase ID tokens by FastAPI, and appropriate authorization checks for protected endpoints and stored records. MongoDB access must be restricted, and queries returning user-specific data must enforce ownership or role-based access controls. URLs, email content, downloaded files, authentication tokens, and other potentially sensitive information must be handled securely. Data collection and retention should be minimized to protect user privacy.

2.5 Design and Implementation Constraints

User Authentication: Firebase Authentication shall be used for user registration, sign-in, and identity verification. The frontend shall use the Firebase Authentication client integration appropriate to the application. Protected requests to FastAPI shall include the Firebase ID token, and the backend shall validate the token server-side before processing the request.

Application Data: MongoDB shall store applicable user profile metadata, scan history, detection results, threat reports, and system records. Authentication credentials and passwords shall remain managed by Firebase Authentication and shall not be stored in MongoDB.

Authorization and Privacy: Data access shall be restricted to the authenticated user or other explicitly authorized roles. The backend shall enforce authorization before reading or changing user-specific MongoDB records. Sensitive URL parameters, email content, webpage contents, downloaded file contents, and authentication tokens shall not be retained unnecessarily. The retention of scan records and reports shall follow the system's configured privacy and retention policy.

Authentication and Application Data Management: Firebase Authentication shall manage user registration, sign-in, and identity verification. MongoDB shall store applicable user profile metadata, scan history, detection results, threat reports, and system records. Authentication credentials and passwords shall not be stored in MongoDB. Data access shall be restricted to authenticated users or explicitly authorized roles.

2.6 User Documentation

The system should provide a user guide, supported-file information, examples of scan workflows, instructions for reviewing reports, explanations of risk decisions, and troubleshooting guidance for failed or unsupported operations.

2.7 Assumptions and Dependencies

The system depends on supported Chromium browser APIs, network connectivity, Firebase Authentication, MongoDB, configured machine-learning models, AI services, and available threat-intelligence services. Detection and prevention capabilities are limited by browser permissions, API availability, model quality, and the evidence observable during browsing.

Chapter 3

External Interface Requirements

3.1 User Interfaces

The browser extension shall provide warnings, risk explanations, and protective-action feedback during browsing. The web dashboard shall provide authentication, manual scan forms for URLs, QR codes, emails, and files, detection-history views, threat reports, analytics, and evidence supporting each security decision.

3.2 Hardware Interfaces

BrowseBlock requires a desktop or laptop computer capable of running a supported Chromium-based browser. Local storage may be used for extension operation and controlled download workflows. No special hardware is required for normal browsing protection.

3.3 Software Interfaces

The system shall interface with Chromium browser-extension APIs, React and Tailwind CSS, FastAPI, Firebase Authentication, MongoDB, XGBoost, Random Forest, file-analysis libraries, QR-decoding components, and configured RAG/LLM or threat-intelligence services.

3.4 Communications Interfaces

Deployed client-server communication shall use HTTPS. Protected requests shall carry Firebase ID tokens, which FastAPI shall verify server-side. API credentials shall not be exposed in extension or dashboard code, and MongoDB shall not be exposed directly to untrusted clients.

Chapter 4

System Features

4.1 URL and Fake QR Phishing

4.1.1 Description and Priority

Evaluates URLs using an XGBoost classifier across normalized numerical features (such as URL length, IP usage, domain age, subdomains, and brand impersonation indicators), and decodes QR codes to evaluate the underlying URL payload to warn or block navigation.

4.1.2 Functional Requirements

REQ-01 (URL Monitoring): The system shall monitor supported browser navigation events and identify URLs requiring phishing analysis.

REQ-02 (URL Normalization): The system shall normalize URLs before extracting features for phishing classification.

REQ-03 (Feature Extraction): The system shall extract configured URL and domain features, including URL length, subdomains, IP-address usage, special characters, suspicious keywords, and available domain information.

REQ-04 (XGBoost Classification): The system shall use an XGBoost classifier to assess the phishing risk of supported URLs.

REQ-05 (QR Code Decoding): The system shall decode QR codes submitted through supported scanning workflows and extract their payloads.

4.1.3 Non-Functional Requirements

REQ-01 (Processing Efficiency): The system shall perform URL preprocessing and classification efficiently to minimize unnecessary browsing delays.

REQ-02 (Detection Accuracy): The URL detector shall be evaluated using appropriate validation data to measure precision, recall, and false-positive rates.

REQ-03 (Input Robustness): The system shall handle malformed, unusually long, and encoded URLs without crashing.

REQ-04 (Privacy): The system shall minimize the storage of sensitive URL parameters and unnecessary browsing information.

4.2 Cross-Site Scripting (XSS)

4.2.1 Description and Priority

Detects potential Cross-Site Scripting (XSS) attacks by inspecting webpage content, JavaScript behavior, and DOM manipulation for suspicious script injection indicators. The system addresses three types of XSS attacks: reflected XSS, stored XSS, and DOM-based XSS. Detection techniques include identifying suspicious URL parameters, injected script patterns, dynamically generated code, obfuscated JavaScript, unsafe DOM sinks,

and unexpected script sources. BrowseBlock uses configured detection rules and observable browser behavior to assess potential threats, warn users, and restrict suspicious interactions where browser capabilities permit.

4.2.2 Functional Requirements

REQ-10 (Webpage Inspection): The system shall inspect supported webpage content, URL parameters, and observable JavaScript behavior for indicators associated with XSS attacks.

REQ-11 (Script Pattern Detection): The system shall identify configured suspicious patterns involving inline scripts, dynamically generated code, obfuscated JavaScript, and unexpected script sources.

REQ-12 (DOM Behavior Analysis): The system shall analyze observable DOM manipulation patterns and potentially unsafe source-to-sink flows associated with client-side script injection.

REQ-13 (Reflected XSS Detection): The system shall identify potential reflected XSS by examining URL parameters and other request-derived inputs for suspicious script injection patterns and their observable reflection in webpage content.

REQ-14 (Stored XSS Detection): The system shall identify observable indicators of potential stored XSS, including suspicious injected content rendered in webpages and persistent script patterns where inspection capabilities permit. Detection shall account for content that may originate from stored user submissions or other server-provided data.

REQ-15 (DOM-Based XSS Detection): The system shall identify potential DOM-based XSS by inspecting observable client-side behavior involving untrusted inputs and potentially unsafe DOM or JavaScript execution sinks.

REQ-16 (XSS Risk Assessment): The system shall evaluate detected indicators using configured rules and available analysis components to determine the potential severity of an XSS threat.

REQ-17 (Reflected XSS Prevention): The system shall warn users about suspected reflected XSS and block known malicious URLs or restrict navigation where supported browser capabilities and reliable detection rules permit.

REQ-18 (Stored XSS Prevention): The system shall warn users about suspected stored XSS content and restrict access to affected pages or known malicious resources where supported browser capabilities permit. The system shall not assume that a browser extension can remove the underlying stored payload from the server.

REQ-19 (DOM-Based XSS Prevention): The system shall warn users about suspected DOM-based XSS and restrict identified malicious navigation or resource loading where supported. The system shall apply available browser-level blocking mechanisms without relying solely on removing DOM elements after potentially harmful scripts have executed.

REQ-20 (XSS Warning): The system shall provide users with a warning when supported inspection identifies potentially dangerous script injection or execution behavior.

REQ-21 (Interaction Protection): The system shall restrict suspicious client-side interactions using supported browser extension capabilities and configured protection rules.

REQ-22 (Detection Logging): The system shall record relevant detection indicators, the identified XSS category where determinable, the risk assessment, and the protective action without unnecessarily storing complete webpage contents.

4.2.3 Non-Functional Requirements

REQ-05 (Performance): The system shall inspect webpage content and observable JavaScript behavior without introducing unnecessary page-loading or interaction delays.

REQ-06 (Detection Reliability): The detector shall minimize false positives involving legitimate JavaScript, URL parameters, dynamic content, and normal DOM manipulation.

REQ-07 (Privacy): The system shall minimize the collection and storage of webpage content, URL parameters, and other potentially sensitive data during XSS detection.

REQ-08 (Detection Coverage): The system shall implement distinct detection rules and test cases for reflected XSS, stored XSS, and DOM-based XSS, subject to browser visibility and extension permission limitations.

REQ-09 (Prevention Limitations): The system shall distinguish between detecting suspicious behavior and preventing script execution, and shall not report an attack as blocked unless the configured protective action has actually been applied.

4.3 Credential Harvesting

4.3.1 Description and Priority

Detects potential credential-harvesting attempts by analyzing suspicious external script loading, deceptive login interfaces, and scam pop-ups. Playwright is used in the analysis and testing workflow to automate webpage interactions, monitor network requests, identify external script domains, and investigate suspicious pop-up behavior. The collected indicators support risk assessment and the development of blocking rules within the BrowseBlock browser extension.

4.3.2 Functional Requirements

REQ-17 (External Script Monitoring): The system shall identify and monitor external domains loading JavaScript resources on webpages using Playwright-based analysis.

REQ-18 (Suspicious Domain Detection): The system shall evaluate external script domains against configured indicators, domain reputation information, and available threat intelligence to identify potentially malicious resources.

REQ-19 (Scam Pop-up Detection): The system shall identify suspicious pop-ups and dynamically generated browser-page overlays that exhibit configured scam indicators.

REQ-20 (Credential-Harvesting Indicators): The system shall analyze observable webpage behavior, including suspicious scripts, deceptive login interfaces, and pop-ups requesting sensitive information, to assess potential credential-harvesting risk.

REQ-21 (Risk Assessment): The system shall combine relevant script-domain, webpage, and pop-up indicators to determine the potential risk of credential harvesting.

REQ-22 (Scam Pop-up Blocking): The browser extension shall warn users about or block detected scam pop-ups using supported browser capabilities and configured detection rules.

REQ-23 (Suspicious Resource Blocking): The system shall block identified malicious script URLs or domains where reliable blocking rules and browser permissions are available.

REQ-24 (Detection Logging): The system shall record detected indicators, risk assessments, and protective actions without collecting or logging actual passwords.

4.3.3 Non-Functional Requirements

REQ-05 (Response Time): The system shall minimize delays during script-domain inspection and scam pop-up detection.

REQ-06 (Credential Privacy): The system shall not collect or log actual passwords during its normal detection process.

REQ-10 (Detection Accuracy): The detector shall minimize false positives involving legitimate third-party scripts, authentication providers, and normal pop-up behavior.

4.4 HTML/JavaScript Redirects

4.4.1 Description and Priority

Monitors HTTP redirects, JavaScript-based navigation, and meta-refresh tags to trace redirect chains, evaluate destination reputation, and interrupt malicious navigation.

4.4.2 Functional Requirements

REQ-25 (HTTP Redirect Monitoring): The system shall monitor supported HTTP redirects and browser navigation events.

REQ-26 (Client-Side Redirect Detection): The system shall inspect observable JavaScript navigation and HTML meta-refresh indicators.

REQ-27 (Redirect Chain Tracking): The system shall maintain an observable sequence of source and destination URLs associated with a redirect chain.

REQ-28 (Destination Reputation Analysis): The system shall evaluate redirect destinations using the URL phishing detector and available threat-intelligence information.

REQ-29 (Redirect Risk Assessment): The system shall identify redirect chains that contain configured indicators of potentially malicious navigation.

REQ-30 (Malicious Navigation Prevention): The system shall warn about or interrupt navigation to high-risk destinations where supported.

REQ-31 (Redirect Event Logging): The system shall record relevant redirect information, the final risk assessment, and the action taken.

4.4.3 Non-Functional Requirements

REQ-11 (Navigation Performance): The system shall analyze redirect chains efficiently to minimize unnecessary navigation delays.

REQ-12 (Monitoring Reliability): The system shall handle incomplete or unavailable redirect information without falsely reporting that the entire redirect chain was analyzed.

REQ-13 (Resource Efficiency): The system shall avoid unnecessary repeated analysis of the same redirect event.

4.5 Clickjacking

4.5.1 Description and Priority

Identifies deceptive framing behavior and suspicious iframe setups to block risky interactions or issue warnings, complementing server-side defenses like CSP frame-ancestors and X-Frame-Options.

4.5.2 Functional Requirements

REQ-32 (Iframe Inspection): The system shall inspect observable iframe attributes and webpage framing structures.

REQ-33 (Suspicious Frame Detection): The system shall identify configured indicators of deceptive framing, suspicious positioning, and potentially misleading interactive elements.

REQ-34 (Interaction Risk Assessment): The system shall evaluate detected indicators to determine potential clickjacking risk.

REQ-35 (Clickjacking Warning): The system shall warn users when supported inspection identifies suspicious framing behavior.

REQ-36 (Interaction Protection): The system shall block or restrict risky interactions where a reliable prevention mechanism is available through supported browser capabilities.

REQ-37 (Detection Logging): The system shall record relevant indicators, risk assessments, and protective actions.

4.5.3 Non-Functional Requirements

REQ-14 (Interaction Performance): The system shall perform frame inspection without significantly affecting normal webpage interactions.

REQ-15 (Detection Reliability): The detector shall minimize false positives involving legitimate iframe usage and embedded content.

REQ-16 (Browser Compatibility): The system shall implement clickjacking protections within the capabilities and permissions of the selected browser.

4.6 Cryptojacking

4.6.1 Description and Priority

Inspects scripts, WebAssembly behaviors, web worker activities, and abnormal resource-usage patterns to identify and block unauthorized cryptocurrency mining in the browser.

4.6.2 Functional Requirements

REQ-38 (Mining Script Detection): The system shall inspect observable script indicators associated with unauthorized cryptocurrency mining.

REQ-39 (WebAssembly and Worker Analysis): The system shall analyze supported WebAssembly and web worker indicators for suspicious mining-related behavior.

REQ-40 (Mining Resource Identification): The system shall identify configured indicators associated with known cryptocurrency-mining resources.

REQ-41 (Resource Usage Analysis): The system shall evaluate available browser resource-usage indicators when supported.

REQ-42 (Cryptojacking Risk Assessment): The system shall combine relevant indicators to determine a cryptojacking risk category.

REQ-43 (Cryptojacking Protection): The system shall warn users about detected risks or block suspicious resources where browser capabilities permit.

REQ-44 (Activity Logging): The system shall record relevant detection evidence and the action taken.

4.6.3 Non-Functional Requirements

REQ-17 (Resource Efficiency): The system shall minimize additional CPU and memory consumption during cryptojacking detection.

REQ-18 (Detection Accuracy): The detector shall minimize false positives involving legitimate computationally intensive webpages.

REQ-19 (Monitoring Limitations): The system shall restrict its conclusions to script behavior and resource information observable through supported browser mechanisms.

4.7 Malicious File Download Detection

4.7.1 Description and Priority

Implements a malicious file download detection pipeline that monitors supported downloads of `.exe`, `.zip`, `.pdf`, `.doc`, and `.docx` files through the browser extension. The system uses content-based file identification, format-specific static analysis, SHA-256 hash calculation, available threat-intelligence checks, deterministic rules, and XGBoost classification to assess file risk without executing untrusted files. The FastAPI backend coordinates analysis and risk assessment, while a native helper application can provide access to file contents and support controlled quarantine and release workflows. Based on the assessment, BrowseBlock shall allow, warn about, cancel, or quarantine files where supported by the selected integration.

4.7.2 Functional Requirements

REQ-45 (Download Monitoring): The browser extension shall monitor supported download events and collect available metadata, including the download URL, filename, MIME type, and download identifier.

REQ-46 (File Type Identification): The system shall identify supported `.exe`, `.zip`, `.pdf`, `.doc`, and `.docx` files using filename extensions, MIME types, and content-based identification where file access is available.

REQ-47 (Executable Analysis): The system shall extract configured executable features, including PE headers, section characteristics, entropy, imported APIs, digital signature information, and resources where applicable.

REQ-48 (PDF Analysis): The system shall inspect PDF structures for potentially suspicious JavaScript, `OpenAction` entries, embedded files, objects, streams, and external

URLs.

REQ-49 (Document Analysis): The system shall inspect supported DOC and DOCX files for applicable macros, OLE objects, embedded files, external relationships, and suspicious document structures.

REQ-50 (Archive Analysis): The system shall inspect ZIP archive metadata, contained file types, suspicious paths, compression characteristics, and nested archives within configured analysis limits.

REQ-51 (Content-Based File Verification): The system shall verify file types using available content-based identification and shall not rely solely on filename extensions or MIME types.

REQ-52 (Feature Extraction): The system shall extract configured numerical and categorical features from supported file formats and produce structured records suitable for classification.

REQ-53 (Hash and Reputation Analysis): The system shall calculate a SHA-256 hash for accessible files and check available threat-intelligence or file-reputation sources where configured.

REQ-54 (File Risk Classification): The system shall evaluate extracted features using a configured XGBoost classifier, deterministic detection rules, and available hash-reputation results.

REQ-55 (Risk Decision): The system shall classify analyzed files as low-risk, suspicious, or high-risk according to the configured decision policy, taking into account analysis completeness and available evidence.

REQ-56 (Download Protection): The browser extension shall warn about or cancel supported downloads according to the risk assessment and available browser capabilities. Where pre-use protection is required, the system shall support a controlled workflow through a native helper application or download gateway.

REQ-57 (File Quarantine and Release): Where a native helper or controlled gateway is deployed, the system shall support holding suspicious files in a controlled location until analysis is complete and shall permit release or quarantine according to the configured risk policy.

REQ-58 (Analysis Coordination): The system shall use the FastAPI backend to coordinate file-analysis requests, feature-processing workflows, classification, and the return of assessment results where backend processing is required.

REQ-59 (Analysis Failure Handling): The system shall identify unsupported formats, malformed files, parser errors, timeouts, and incomplete analysis, and shall apply the configured warning or hold policy rather than automatically treating failed analysis as a safe result.

REQ-60 (File Analysis Logging): The system shall record the file type, timestamp, available hash, risk assessment, relevant detection indicators, and protective action, subject to configured privacy and retention policies.

4.7.3 Non-Functional Requirements

REQ-20 (Safe File Processing): The system shall analyze untrusted files without executing them and shall isolate file parsers and analysis components where practical.

REQ-21 (Resource Limits): The system shall enforce configurable limits on file size, processing time, archive depth, decompressed size, entry count, and resource consumption.

REQ-22 (Processing Reliability): The system shall handle malformed files, unsupported formats, parser failures, and analysis timeouts without crashing the extension, backend, or analysis service.

REQ-23 (Privacy and Retention): The system shall minimize the transmission and retention of downloaded file contents, retaining only the information required for detection, auditing, and configured security purposes.

REQ-24 (Type Verification): The system shall use content-based file identification where file bytes are available and shall treat discrepancies between the filename extension, MIME type, and actual file format as potential risk indicators rather than definitive proof of maliciousness.

REQ-30 (Analysis Isolation): The system shall isolate file-processing components from the main application where practical and shall prevent untrusted files from being executed during static analysis.

REQ-31 (Protection Reliability): The system shall distinguish between download monitoring, file analysis, and enforced blocking or quarantine, and shall report a protective action as successful only when the action has actually been confirmed.

REQ-32 (Model Reliability): The system shall evaluate the XGBoost classifier using appropriate validation data and shall not rely on model predictions alone when making high-impact file-blocking decisions.

REQ-33 (Integration Compatibility): The system shall implement download monitoring and enforcement according to the capabilities and permissions of the selected browser APIs, native helper, or controlled download gateway.

4.8 Email Phishing Detection

4.8.1 Description and Priority

Implements a two-stage email phishing detection pipeline. The browser extension performs lightweight initial screening using a Random Forest classifier and available email-authentication signals, including SPF, DKIM, and DMARC results. For detailed investigation, the web dashboard uses a Retrieval-Augmented Generation (RAG) pipeline to analyze email content, identify social engineering tactics, and extract URLs. The extracted URLs are evaluated using BrowseBlock's URL phishing detection module. This architecture enables efficient initial screening in the browser extension while providing deeper contextual analysis through the web dashboard when required.

4.8.2 Functional Requirements

REQ-56 (Email Input): The system shall accept email content through supported browser extension integrations or manual submission workflows in the web dashboard.

REQ-57 (Email Feature Extraction): The system shall extract configured email features, including message length, subject characteristics, URL count, attachment presence, suspicious keyword counts, and sender-domain mismatch, where the required data is available.

REQ-58 (Random Forest Screening): The browser extension shall use a Random Forest classifier to perform lightweight initial email phishing assessment using configured email features.

REQ-59 (Email Authentication Signals): The browser extension shall incorporate available SPF, DKIM, and DMARC authentication results into the initial email risk assessment.

REQ-60 (Initial Risk Assessment): The browser extension shall combine the Random Forest prediction and available email-authentication signals to identify potentially suspicious emails and provide an initial risk assessment.

REQ-61 (Email Submission for Detailed Analysis): The system shall allow users to submit email content to the web dashboard for detailed phishing analysis when required.

REQ-62 (RAG-Based Email Analysis): The web dashboard shall use a Retrieval-Augmented Generation (RAG) pipeline to retrieve relevant information from a configured phishing knowledge base and support the contextual analysis of submitted email content.

REQ-63 (Email URL Extraction): The RAG-based analysis workflow shall identify and extract supported URLs from submitted email content for further security assessment.

REQ-64 (URL Phishing Analysis): The system shall evaluate extracted URLs using BrowseBlock's URL phishing detection module and incorporate the resulting risk information into the detailed email assessment.

REQ-65 (Semantic Threat Assessment): The configured LLM component within the RAG workflow shall analyze email content for indicators of social engineering, impersonation, urgency, credential harvesting, and malicious intent.

REQ-66 (Detailed Risk Classification): The web dashboard shall combine available initial screening results, URL analysis results, and contextual analysis evidence to classify the email as safe, suspicious, or high-risk according to the configured decision policy.

REQ-67 (Email Warning): The system shall warn users about suspicious or high-risk emails and present available supporting evidence.

REQ-68 (Email Link Protection): The system shall warn users about or restrict access to associated email links and actions where supported by the browser integration.

REQ-69 (Email Event Logging): The system shall record the assessment result, timestamp, available evidence, and protective action, subject to configured privacy and retention policies.

4.8.3 Non-Functional Requirements

REQ-25 (Analysis Efficiency): The system shall perform lightweight initial screening within the browser extension and reserve resource-intensive RAG and LLM analysis for detailed investigations through the web dashboard.

REQ-26 (Processing Timeouts): The system shall apply configurable timeouts to RAG retrieval, LLM inference, and external service requests.

REQ-27 (Email Privacy): The system shall minimize the transmission and retention of sensitive email content and obtain user authorization where required before submitting email content for detailed analysis.

REQ-28 (Authentication Signal Reliability): The system shall not treat successful SPF, DKIM, or DMARC checks as definitive proof that an email is safe.

REQ-29 (Failure Handling): The system shall distinguish incomplete or failed email analysis from a completed safe verdict and shall communicate analysis limitations where appropriate.

4.9 Browser Notification Scams

4.9.1 Description and Priority

Detects deceptive notification requests and prompts imitating urgent system alerts, prizes, or fake security warnings, issuing warnings or blocking interactions when browser APIs allow.

4.9.2 Functional Requirements

REQ-68 (Notification Request Monitoring): The system shall monitor supported webpage notification-permission requests and related prompts.

REQ-69 (Deceptive Prompt Detection): The system shall identify configured indicators of fake security warnings, prize scams, urgent alerts, and deceptive notification requests.

REQ-70 (Notification URL Analysis): The system shall evaluate relevant URLs associated with detected notification-related scams.

REQ-71 (Notification Scam Assessment): The system shall assess observable notification-related indicators and determine a risk category.

REQ-72 (Notification Protection): The system shall warn users about suspicious notification-related interactions and restrict associated actions where browser capabilities permit.

REQ-73 (Notification Event Logging): The system shall record relevant detection evidence and protective actions.

4.9.3 Non-Functional Requirements

REQ-30 (Usability): The system shall minimize interference with legitimate website notification functionality.

REQ-31 (Detection Reliability): The system shall minimize false positives involving legitimate notification prompts.

REQ-32 (Monitoring Scope): The system shall limit its conclusions to notification-related content and behavior observable through supported browser mechanisms.

4.10 Browser Extension Abuse

4.10.1 Description and Priority

Identifies suspicious browser extensions by monitoring overly broad permissions, unexpected webpage tampering, and malicious script injection through observable browser security mechanisms.

4.10.2 Functional Requirements

REQ-74 (Extension Behavior Monitoring): The system shall monitor observable browser and webpage indicators associated with potential extension abuse.

REQ-75 (Permission Analysis): The system shall evaluate extension-permission information where the browser exposes the required information through supported APIs.

REQ-76 (Webpage Tampering Detection): The system shall identify configured indicators of unexpected webpage manipulation and suspicious script injection.

REQ-77 (Abuse Risk Assessment): The system shall classify observed indicators using configured detection rules and available evidence.

REQ-78 (Abuse Warning): The system shall warn users about potentially abusive extension behavior when sufficient observable evidence is available.

REQ-79 (Abuse Event Logging): The system shall record available indicators and the resulting risk assessment.

4.10.3 Non-Functional Requirements

REQ-33 (Monitoring Efficiency): The system shall minimize additional browser resource consumption during extension-abuse detection.

REQ-34 (Detection Scope): The system shall limit its conclusions to extension behavior and permission information observable through supported browser mechanisms.

REQ-35 (Privacy): The system shall minimize the collection of unrelated browsing information during extension-abuse detection.

Chapter 5

Other Nonfunctional Requirements

5.1 Performance Requirements

The extension shall provide responsive protection under normal browsing conditions. Resource-intensive analysis should be delegated to the backend or dashboard, and long-running scans shall expose their status. The system shall handle timeouts and component failures without leaving users with an unknown or misleading result.

5.2 Safety Requirements

The system shall not execute untrusted downloaded files for analysis. It shall distinguish detection from confirmed prevention, report incomplete analysis, communicate uncertainty, and avoid treating automated checks or AI explanations as guarantees of safety.

5.3 Security Requirements

The deployed system shall use HTTPS, Firebase Authentication, server-side token verification, authorization checks, restricted database access, secure handling of URLs and uploaded content, and configured retention controls. API keys and database credentials shall not be exposed in frontend code or public repositories.

5.4 Software Quality Attributes

BrowseBlock should provide usability, reliability, maintainability, scalability, interoperability, transparency, privacy, and compatibility. The system shall provide understandable errors, preserve user control over consequential actions, and report the limitations of models and browser APIs.

Chapter 6

Other Requirements

The system shall support future improvements such as additional threat categories, updated machine-learning models, new browser APIs, improved threat intelligence, and additional dashboard analytics. The architecture should remain modular so that detection modules, model versions, integrations, and retention policies can be tested and updated independently.

Chapter A

Glossary

Browser Extension: The BrowseBlock client component that monitors supported browser activity and provides warnings or protective actions.

Risk Assessment: The evaluation of available evidence to classify activity as Safe, Suspicious, or High-risk.

RAG: Retrieval-Augmented Generation, which supplies retrieved information to an AI model for contextual analysis.

XSS: Cross-Site Scripting, a class of attacks involving the injection or execution of untrusted scripts.

SRS: Software Requirements Specification.

Chapter B

Analysis Models

The final report may include use-case, system-architecture, activity, and data-flow diagrams showing users, browser-extension events, backend analysis, AI/ML components, databases, and protective actions.

Chapter C

To Be Determined List

The following items shall be finalized during implementation: supported browser versions and APIs, model versions and evaluation datasets, threat-intelligence providers, supported file-size limits, retention periods, deployment topology, and the exact set of features included in the first prototype.